UADE

Clase 3 - Temario

- Listas / Ampliación de funcionalidades
- Cadenas de caracteres

Autoevaluación

COPIAR CONTENIDO DE UNA LISTA

- PARA REALIZAR LA COPIA DE UNA ACCIÓN SE REALIZA A TRAVÉS DEL METODO COPY()
- COMO HEMOS VISTO, EL SIGNO DE IGUALDAD SE PUEDE UTILIZAR PARA CONSTRUIR UN DUPLICADO DE UNA LISTA. SIN EMBARGO, LA NUEVA LISTA ESTARÁ VINCULADA A LA EXISTENTE. ESTO SIGNIFICA QUE LA NUEVA LISTA TAMBIÉN SE VERÁ ALTERADA SI SE ACTUALIZA LA LISTA ORIGINAL

Copiar una lista

```
lista=[5,258,369]

lista2 = lista.copy()

print(lista,lista2)
```

ORDENAR UNA LISTA

- LAS LISTAS SON SECUENCIAS ORDENADAS. ESTO QUIERE DECIR QUE SUS ELEMENTOS SIEMPRE SE DEVUELVEN EN EL MISMO ORDEN EN QUE FUERON AÑADIDOS.

- ES POSIBLE ORDENAR LOS ELEMENTOS DE UNA LISTA CON EL MÉTODO SORT(), QUE ORDENA LOS ELEMENTOS DE LA LISTA Y MODIFICA LA LISTA ACTUAL (NO SE OBTIENE UNA NUEVA LISTA).

- PARA OBTENER ORDEN DE FORMA DESCENDENTE, SE DEBE ESPECIFICAR EL PARÁMETRO REVERSE=TRUE EN EL LLAMADO.

- ORDENA LAS PALABRAS EN ORDEN LEXICOGRÁFICO. COMO UN DICCIONARIO

ORDENAR UNA LISTA

```
numeros = [3, 11, 6, 1, 7, 8]
numeros.sort() #Orden ascendente
print(numeros)
numeros.sort(reverse=True) #Orden descendente
print(numeros)
```

- **DIFERENCIA ENTRE MAYÚSCULAS Y MINÚSCULAS. USA VALORES INTERNOS ASCII**

REBANADO

- **SEGMENTACIÓN DE LISTAS**
  - PARA ACCEDER A UN RANGO DE ELEMENTOS EN UNA LISTA ES NECESARIO DIVIDIR LA MISMA.
  - PARA ELLO SE UTILIZA EL OPERADOR DE CORTA DOS PUNTOS (:)
  - LA SEGMENTACIÓN DEVUELVE UNA LISTA NUEVA A PARTIR DE LA LISTA EXISTENTE SIN ALTERAR LA ORIGINAL

0	1	2	3	4	5
o	l	i	v	e	r
-6	-5	-4	-3	-2	-1

lista[inicio:fin:paso]

paso es opcional

INCLUYE INICIO
NO INCLUYE FIN

REBANADO - EJEMPLOS

0	1	2	3	4	5
o	l	i	v	e	r
-6	-5	-4	-3	-2	-1

ORIGINAL

letras[-6:-3]

o	l	i

letras[1:5]

l	i	v	e

letras[-2:]

e	r

letras[:3]

o	l	i

letras[:-3]

o	l	i

letras[3:]

v	e	r

letras[::-1]

r	e	v	i	l	o

REBANADO

EL TERCER PARÁMETRO ES EL PASO
POR DEFECTO ES 1, PERO SE PUEDE MODIFICAR

```
print(vocales[0:4:1])
```
→ #Esto es equivalente a vocales[0:4]

```
print(vocales[0:4:2])
```
→ Comienza en el 0 y avanza 2 pasos, luego otros 2.

```
print(vocales[::-1])
```
→ Invertir lista. Si queremos invertir la lista podemos avanzar con el paso -1

REVERSE Y REVERSED

- SE UTILIZAN PARA INVERTIR EL ORDEN DE LOS ELEMENTOS DE UNA LISTA DE FORMA PERMANENTE
- A DIFERENCIA DEL REBANADO QUE NO ALTERA LA LISTA ORIGINAL, **EL MÉTODO REVERSE() MODIFICA LA LISTA ORIGINAL**
- EL **MÉTODO REVERSE() LA LISTA ORIGINAL SE MODIFICA**, SI QUEREMOS INVERTIR **LA LISTA SIN MODIFICAR LA ORIGINAL SE DEBE UTILIZAR REVERSED()** EN CONJUNTO CON LA FUNCIÓN LISTA

```
lista = [3, 6, 9, 12, 15]
#Aplicamos el método reverse()
lista.reverse()
#Imprimimos la lista invertida
print(lista) #[15, 12, 9, 6, 3]
lista = [3, 6, 9, 12, 15]
```

```
#Creamos una nueva lista invertida
listaInvertida = list(reversed(lista))
#Imprimimos la lista invertida
print(listaInvertida) #[15, 12, 9, 6, 3]
#La lista original no se ha modificado
print(lista) #[3, 6, 9, 12, 15]
```

MÉTODO INSERT

AGREGA UN ELEMENTO EN EL ÍNDICE DADO DE UNA LISTA EXISTENTE. ACEPTA DOS PARÁMETROS, EL ÍNDICE A INSERTAR Y EL VALOR A INSERTAR

vocales.insert(2,'E') → Agrega el valor E en la posicion 2

vocales.insert(0,'E') → Agrega el valor E en la posición inicial

vocales.insert(len(vocales),'E') → Agrega el valor E en la posición final

MÉTODO EXTEND

NOS PERMITE AÑADIR ELEMENTOS DE UNA LISTA A UNA LISTA EXISTENTE, ITERANDO SOBRE EL ARGUMENTO Y AGREGANDO CADA ELEMENTO A LA LISTA

DEBEMOS USAR ESTE MÉTODO SI NECESITAMOS AGREGAR VARIOS ELEMENTOS A UNA LISTA COMO ELEMENTOS INDIVIDUALES

POR TANTO, EL ARGUMENTO DADO DEBE SER DE TIPO ITERABLE. POR EJEMPLO OTRA LISTA.

```
vocales.extend(['a','e','i','o','u'])
```

→ Extiende la lista, agregandole otra lista de vocales

fibonacci

0	1	1	2	3

MÉTODO LEN DEVUELVE LA LONGITUD DE LA LISTA

len(fibonacci) → Devuelve 5

MÉTODO COUNT DEVUELVE LA CANTIDAD DE VECES QUE APARECE UN ELEMENTO EN UNA LISTA

fibonacci.count(1) → Devuelve 2

MÉTODO SUM RETORNA LA SUMA DE TODOS LOS ELEMENTOS EN LA LISTA

sum(fibonacci) → Devuelve 7

fibonacci

0	1	1	2	3

MÉTODO MIN DEVUELVE EL VALOR MINIMO DE UNA LISTA

min(fibonacci) → Devuelve 0

MÉTODO MAX DEVUELVE EL VALOR MÁS ALTO DE UNA LISTA

max(fibonacci) → Devuelve 3

OPERADOR IN

UN OPERADOR DE PERTENENCIA SE EMPLEA PARA IDENTIFICAR LA EXISTENCIA DE UN VALOR DENTRO DE UNA LISTA. RETORNA TRUE EN CASO DE QUE EXISTA; Y FALSE EN CASO CONTRARIO.

TAMBIÉN PODEMOS UTILIZAR SU NEGACIÓN ANTEPONIENDO EL OPERADOR NOT, INVIRTIENDO SU RESULTADO DE VERDAD.

```
lista = [1, 3, 5, 8, 13]
if (8 in lista):
    print("El valor se encuentra en la lista")
else:
    print("El valor no se encuentra en la lista")

if (7 not in lista):
    lista.append(7) #Agrega el elemento si no existe
```

FOR-EACH

OTRA FORMA DE UTILIZAR EL OPERADOR IN ES EL RECORRIDO DE ELEMENTOS DE UNA LISTA UTILIZANDO LA ESTRUCTURA DE BUCLE FOR, SIN LA NECESIDAD DE ACCEDER POR SUS ÍNDICES; DONDE CADA UNO DE LOS ELEMENTOS DE LA LISTA SE DEPOSITA EN LA VARIABLE A LA IZQUIERDA DEL IN EN CADA ITERACIÓN.

```python
lista = [1, 3, 5, 8, 13]
for numero in lista:
    print(numero)

for i in range(0, len(lista)):
    print(lista[i])
```

COMPRENSIÓN DE LISTAS

ES UNA CONSTRUCCIÓN SINTÁCTICA PARA CREAR LISTAS A PARTIR DE LOS ELEMENTOS DE OTROS ELEMENTOS ITERABLES (COMO OTRAS LISTAS) DE UNA FORMA RÁPIDA DE ESCRIBIR, MUY LEGIBLE Y FUNCIONALMENTE EFICIENTE.

[expresión for elemento in colección if condición]

Expresion → transformación del elemento

Bucle for

```
lista = []
for i in range(1, 101):
    lista.append(i)

print(lista)
```

Es opcional

```
lista = [ i for i in range (1, 101) ]
print(lista)
```

COMPRENSIÓN DE LISTAS

PASEMOS A OTRO EJEMPLO EN EL QUE SE RECORRE UNA LISTA EXISTENTE Y, MEDIANTE LA MISMA, SE DESEA CREAR UNA NUEVA, PERO CON LOS VALORES ELEVADOS AL CUADRADO.

```
lista = [1, 2, 3, 4, 5]
listaCuadrados = []
for elemento in lista:
    listaCuadrados.append(elemento ** 2)
```

```
lista = [1, 2, 3, 4, 5]
listaCuadrados = [ elemento ** 2 for elemento in lista ]
```

COMPRENSIÓN DE LISTAS

CREAR LA NUEVA LISTA CON LOS ELEMENTOS PARES DE LA LISTA ORIGINAL.

```
lista = [1, 2, 3, 4, 5]
listaCuadrados = [ elemento ** 2 for elemento in lista if elemento % 2 == 0 ]
```

CADENAS DE CARACTERES

LAS CADENAS DE CARACTERES O STRING SON UNA SECUENCIA DE CARACTERES QUE REPRESENTAN TEXTO. SON DEL TIPO STR Y SE DELIMITAN CON COMILLAS SIMPLES O DOBLES

```
print("Esto es una cadena en 'python'")
print('Esto es una cadena en \'python\'')
```

CADENA DE ESCAPE ES UNA SECUENCIA DE CARACTERES QUE COMIENZA CON UNA BARRA INVERTIDA Y QUE REPRESENTA UN CARÁCTER ESPECIAL EN UNA CADENA DE TEXTO

Secuencia Escape	Significado
\\	Backslash (\)
\'	Comillas simples (')
\"	Comillas dobles (")
\n	Salto de línea
\t	Tabulación Horizontal (TAB)

CADENAS DE CARACTERES

CONCATENACIÓN PERMITE UNIR DISTINTAS CADENAS MEDIANTE EL USO DEL **+**

```
cadena1 = "Hola"
cadena2 = "Mundo"
cadena3 = cadena1 + " " + cadena2
print(cadena3) #Hola Mundo
```

REPLICAR PERMITE REALIZAR DETERMINADO NÚMERO DE COPIAS

```
cadena = "Hola mundo"
print(cadena*3) #Hola mundoHola mundoHola mundo
```

TRANSFORMACIÓN DE CARACTERES

UPPER - MAYÚSCULAS

```
cadena = "Hola mundo"
print(cadena.upper()) #HOLA MUNDO
```

UPPER - MINÚSCULAS

```
print(cadena.lower()) #hola mundo
```

SWAPCASE - INVERSIÓN DE MAYÚSCULAS Y MINÚSCULAS

```
print(cadena.swapcase()) #hOLA mUNDO
```

TITLE - CAPITALIZADO

```
cadena = "hola mundo"
print(cadena.title()) #Hola Mundo
```

TRANSFORMACIÓN DE CARACTERES

REPLACE - REEMPLAZO DE CARACTERES

```
cadena = "Hola mundo"
print(cadena.replace("Hola", "Chau")) #Chau Mundo
```

JOIN - UNIÓN DE UNA LISTA

```
nombres = ['Oliver', 'Irina', 'Gabriela']
cadenaUnida = ', '.join(nombres)
print(cadenaUnida) #Oliver, Irina, Gabriela
```

LSTRIP, RSTRIP Y STRIP QUITADO DE ESPACIOS AL COMIENZO O FIN DE LA CADENA

```
cadena = "        Hola mundo        "
print(cadena.lstrip()) #'Hola mundo        '
print(cadena.rstrip()) #'        Hola mundo'
print(cadena.strip()) #'Hola mundo'
```

TRANSFORMACIÓN DE CARACTERES

SPLIT
DIVISIÓN DE CADENA DE CARACTERES

```
cadena = "Hola mundo; chau mundo"
print(cadena.split()) #['Hola', 'mundo;', 'chau', 'mundo']
print(cadena.split(";")) #['Hola mundo', ' chau mundo']
```

f-string

SON CADENAS DE TEXTO CON UNA F AL INICIO Y EXPRESIONES ENTRE LLAVES QUE SE SUSTITUYEN POR SUS VALORES, TAL Y COMO SE MUESTRA EN EL SIGUIENTE EJEMPLO:

```
nombre = "Oliver"
apellido = "Pérez"
dni = "59.111.333"

#Nombre: Oliver - Apellido: Pérez - DNI 59.111.33
print(f"Nombre: {nombre} - Apellido: {apellido} - DNI {dni}")
```

Guía de ejercicios Listas

Ejercicio 1

Una escuela está desarrollando un sistema para generar reportes automáticos de desempeño.

Cada alumno tiene una nota final en la materia.

notas = [4, 7, 2, 9, 6, 10, 3]

El sistema necesita mostrar en el panel únicamente a los alumnos que aprobaron la materia (nota mayor o igual a 6), ya que estos serán incluidos en el cuadro de honor.

Generar una nueva lista con las notas aprobadas.

Guía de ejercicios Listas

Ejercicio 2

Una tienda online quiere aplicar automáticamente un descuento del 10% a todos sus productos durante una campaña.

precios = [1000, 2500, 3000, 1500]

El sistema debe calcular los nuevos precios sin modificar la lista original.

Generar una lista con los precios con descuento aplicado.

Guía de ejercicios Listas

Ejercicio 3

Una red social detectó que los nombres de usuario se almacenaron en minúscula, pero el sistema de visualización requiere mostrarlos en mayúscula.

usuarios = ["ana", "luis", "pedro", "carla"]

Generar una lista nueva con los nombres en mayúscula.

Guía de ejercicios Listas

Ejercicio 4

Un sensor IoT registra valores cada minuto.

El dashboard solo muestra los últimos 3 valores registrados.

registros = [10, 12, 15, 14, 18, 20, 22]

Obtener los últimos 3 valores usando slicing.

Guía de ejercicios Listas

Ejercicio 4

Un sensor IoT registra valores cada minuto.

El dashboard solo muestra los últimos 3 valores registrados.

registros = [10, 12, 15, 14, 18, 20, 22]

Obtener los últimos 3 valores usando slicing.